and Cypress (https://www.cypress.io) are two of the most popular modern tools for UI testing.

ASP.NET Core has a couple of tricks up its sleeve when it comes to integration testing, as you'll see in chapter 36. You can use the Test Host package to run an in-process ASP.NET Core server, which you can send requests to and inspect the responses. This saves you from the orchestration headache of trying to spin up a web server on a different process, making sure ports are available, and so on, but still allows you to exercise your whole app.

At the other end of the scale, the Entity Framework Core (EF Core) SQLite in-memory database provider lets you isolate your tests from the database. Interacting with and configuring a database is often one of the hardest aspects of automating tests, so this provider lets you sidestep the problem. You'll see how to use it in chapter 36.

The easiest way to get to grips with testing is to give it a try, so in the next section you'll create your first test project and use it to write unit tests for a simple custom service.

35.2 Creating your first test project with xUnit

As I described in section 35.1, to create a test project you need to use a testing framework. You have many options, such as NUnit and MSTest, but (anecdotally) the most used test framework with ASP.NET Core is xUnit (https://xunit.net). The ASP.NET Core framework project itself uses xUnit as its testing framework, so it's become somewhat of a convention. If you're familiar with a different testing framework, feel free to use that instead.

Visual Studio includes a template to create a .NET 7 xUnit test project, as shown in figure 35.1. Choose **File** > **New** > **Project**, and choose **xUnit Test Project** in the New Project dialog box. Alternatively, you could choose